Surrogate-Assisted Evolutionary Deep Learning Using an End-to-End Random Forest-Based Performance Predictor

Yanan Sun, *Member, IEEE*, Handing Wang, *Member, IEEE*, Bing Xue, *Member, IEEE*, Yaochu Jin, *Fellow, IEEE*, Gary G. Yen, *Fellow, IEEE*, and Mengjie Zhang, *Fellow, IEEE*

***Abstract*—Convolutional neural networks (CNNs) have shown remarkable performance in various real-world applications. Unfortunately, the promising performance of CNNs can be achieved only when their architectures are optimally constructed. The architectures of state-of-the-art CNNs are typically handcrafted with extensive expertise in both CNNs and the investigated data, which consequently hampers the widespread adoption of CNNs for less experienced users. Evolutionary deep learning (EDL) is able to automatically design the best CNN architectures without much expertise. However, the existing EDL algorithms generally evaluate the fitness of a new architecture by training from scratch, resulting in the prohibitive computational cost even operated on high-performance computers. In this paper, an end-to-end offline performance predictor based on the random forest is proposed to accelerate the fitness evaluation in EDL. The proposed performance predictor shows the promising performance in term of the classification accuracy and the consumed computational resources when compared with 18 state-of-the-art peer competitors by integrating into an existing EDL algorithm as a case study. The proposed performance predictor is also compared with the other two representatives of existing performance predictors. The experimental results show the proposed performance predictor not only significantly speeds up the fitness evaluations but also achieves the best prediction among the peer performance predictors.**

***Index Terms*—Convolutional neural network (CNN), evolutionary deep learning (EDL), performance predictor, random forest, surrogate model.**

Manuscript received January 18, 2019; revised April 6, 2019 and May 15, 2019; accepted June 7, 2019. Date of publication June 24, 2019; date of current version March 31, 2020. This work was supported in part by the National Natural Science Foundation of China under Grant 61803277, in part by the Fundamental Research Funds for the Central Universities, in part by the National Natural Science Fund of China for Distinguished Young Scholar under Grant 61625204, in part by the Marsden Fund of New Zealand Government under Contract VUW1209, Contract VUW1509, and Contract VUW1615, in part by the Huawei Industry Fund under Grant E2880/3663, and in part by the University Research Fund at Victoria University of Wellington under Grant 209862/3580 and Grant 213150/3662. *(Corresponding authors: Handing Wang; Bing Xue.)*

Y. Sun is with the College of Computer Science, Sichuan University, Chengdu 610065, China, and also with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: ysun@scu.edu.cn).

H. Wang is with the School of Artificial Intelligence, Xidian University, Xi'an 710071, China (e-mail: hdwang@xidian.edu.cn).

B. Xue and M. Zhang are with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: bing.xue@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

Y. Jin is with the Department of Computer Science, University of Surrey, Guildford GU2 7XH, U.K. (e-mail: yaochu.jin@surrey.ac.uk).

G. G. Yen is with the School of Electrical and Computer Engineering, Oklahoma State University, Stillwater, OK 74078 USA (e-mail: gyen@okstate.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TEVC.2019.2924461

I. Introduction

CONVOLUTIONAL neural networks (CNNs) [1], [2], as the most dominant deep learning approaches [3], have been demonstrating their promising performance in addressing the various real-world applications, such as image classification [4], speech understanding [5], and natural language processing [6], to name a few. It is well known that the performance of CNNs highly relies upon their architectures, and a new architecture must be redesigned if the addressed data has been changed. Unfortunately, designing architecture with the best performance for the investigated data requires extensive expertise in both the CNNs and the data domain [7], which is not necessarily held by the interested users. Designing the best CNN architecture for the given data can be seen as an optimization problem, which can be mathematically formulated by

$$\arg\max_{A_\lambda} \ \mathcal{L}(A_\lambda, \mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}) \ \text{s.t.} \ \lambda \in \Lambda \tag{1}$$

where λ refers to the parameters related to the architectures of CNNs, such as the number of convolutional layers and the configurations of pooling layers; Λ refers to the parameter space, A_λ denotes the CNN algorithm A adopting the architecture parameters λ, and $\mathcal{L}$ means the performance measure of A_λ on the test data D_{test} after it has been trained on the training data $\mathcal{D}_{\text{train}}$. Generally, λ is with the discrete values, e.g., the configurations of both convolutional and pooling layers must be integers, which results in the architecture optimization not being well-addressed by the exact optimization algorithms. To address this challenge, researchers have developed algorithms (e.g., [8]–[14]) to design the best CNN architecture by effectively solving the problems represented in (1). Among these algorithms, the evolutionary algorithm-based ones, which are called the evolutionary deep learning algorithms (EDLs), are much preferred because EDLs generally

1089-778X © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

consume less computational resource than other CNN architecture design algorithms [11]–[14], while still can achieve the promising results. For example, on the CIFAR10 dataset [15], the evolutionary algorithm-based automatic evolving CNN (AE-CNN) algorithm finds the best CNN architectures by consuming 22 days using three graphics processing units (GPUs), and achieves the classification error rate of 4.7%. However, the reinforcement learning-based neural architecture search method [9] consumes 28 days using 800 GPUs, while achieves the classification error rate of 6.01% which is a worse result than AE-CNN.

Evolutionary algorithm is a type of population-based heuristic computational paradigms [16]–[18], and has been widely used in addressing various complex optimization problems [19]–[21], mainly because of its gradient free and insensitiveness to local minimum [22]–[24]. In EDLs, each individual is transformed to a CNN with the corresponding architecture through the mapping from genotype to phenotype, and then the weights of the CNN are initialized and iteratively trained based on $\mathcal{D}_{\text{train}}$ commonly with stochastic gradient descent (SGD) [25], [26] given a number of epochs. During each epoch, the CNN also needs to be trained with multiple iterations in which the number is determined by the adopted batch size and the size of $\mathcal{D}_{\text{train}}$. Such a single fitness evaluation typically takes several hours to days. As a result, the potential promising architecture may not be found through EDLs within a limited computational budget due to the time-consuming fitness evaluation. This will become even more severe in the case of a large population and with a large number of generations, which is a more widely used configuration of evolutionary algorithms targeting at finding the better solutions [16], [18], [27]. Therefore, speeding up the fitness evaluation of CNN is essential in the design of EDL.

A common way to accelerate the fitness evaluation of CNN is to use a performance predictor [28]. In 2014, Swersky *et al.* proposed the Freeze-thaw Bayesian optimization (FBO) algorithm [29] by using a Gaussian process regression [16], which built the model based on the training performance in the first t epochs to predict performance at the T ($T > t$) epoch. Followed by the same routine, the learning curve extrapolation methods in [30] and [31] employed a set of functions from exponential families and the Bayesian neural network, respectively, to build the model based on the training performance in the first t epochs. Unfortunately, these mentioned performance predictors are typically based on the expensive Markov chain Monte Carlo sampling procedures and manually designed curve functions, which are with high computational complexity. In 2017, Deng *et al.* proposed the Peephole algorithm [32] that predicted the performance of a CNN based on its architecture information, where a large number of different CNNs were trained, and the performance of all sampled networks in each epoch was collected as the training data, and then a long-short term memory neural network [33] is used to train the model. In addition, the trainless accuracy predictor (TAP) [34] and the accelerating neural network architecture search (ANNAS) [35] algorithms also employed similar methods to predict the performance of the given CNNs.

In fact, the optimization problem in EDLs is a type of computationally expensive problems, which is usually solved by surrogate-assisted evolutionary algorithms (SAEAs) [36], using cheap approximated regression and classification models (like Gaussian process model [37], radial basis function network [38], etc.) to replace the expensive fitness evaluation [39]. SAEAs have shown an effectiveness and efficiency in various real-world optimization applications [40]. Generally, the models are trained from a small number of expensive fitness evaluations (i.e., the samples with evaluated objective values), and then the trained models are used as the fitness predictors in the evolutionary search to accelerate the optimization process [41]. According to [42], SAEAs can be divided into online and offline algorithms, which depend on whether the expensive fitness evaluations are used to enrich the training data during the optimization process. Although new training data in online algorithms can significantly improve the quality of the model and optimization performance, offline algorithms are more practical than the online algorithms due to the hardness and high cost in obtaining new data [40]. Without sampling new training data, offline algorithms can be very fast, and its data collection and optimization search are completely separated. So far, the base algorithms commonly applied in offline SAEAs are data preprocessing [43], data mining [42], and ensemble learning [44], since they can effectively and robustly address the problems having limited training data.

Our goal in this paper is to present an effective and efficient end-to-end performance predictor (E2EPP) based on random forest [45]. The adoption of random forest lies in its effectiveness to a limited number of training samples, applicability to discrete variables, and robustness to its parameter settings [46], [47]. The proposed E2EPP is able to directly know the performance of a CNN once it looks at the new CNN architectures, which is naturally able to speed up the evolutionary process of EDLs. The contributions of this paper are summarized below.

1) The training data of the random forest are a set of data pairs, and each pair is composed of the CNN architecture and its performance. However, the architecture is modeled by describing languages, e.g., a paragraph of words, which cannot be directly used as the input to the random forest. Therefore, we propose an effective encoding method that is capable of extracting the features of CNN architectures as the numerical values, which are further used as the samples for training the random forest. As a case study, AE-CNN [48], as the representative of the state-of-the-art EDLs, is adopted to investigate the details of encoding.
2) The trained random forest is based on a set of samples of CNN architectures (and their performance) which do not necessarily cover all the architecture space. If the performance prediction is adopted from the tree giving the best performance, the obtained result may be biased. To this end, we employ a selective ensemble strategy to choose diverse trees for combination in the local area of every generation.
3) Considering the two goals in term of using the performance predictor: a) the efficiency in speeding up

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

the evolutionary process and b) the effectiveness in finding the best CNN architecture, we perform extensive experiments to verify these aspects.

The remainder of this paper is organized as follows. The background and related work are introduced in Section II. This is followed by the algorithm details of the proposed random forest-based performance predictor in Section III. In order to validate the effectiveness and the efficiency of the proposed performance predictor, the experiment designs, and the experimental results are presented in Sections IV and V, respectively. Finally, the conclusion and future works are described in Section VI.

II. Literature Review

In this section, the EDL, AE-CNN, and random forest are first introduced as the base algorithms in Section II-A, which is helpful to know the details of the proposed performance predictor. Then, the related work to performance predictors is presented in Section II-B, for better justifying the effectiveness and efficiency of the proposed work.

A. Background

1) EDL: A general framework of EDLs [11]–[14] is composed of the steps shown below.

- Step 1: Randomly initialize a population with the predefined size based on the corresponding genotype-to-phenotype mapping strategy.
- Step 2: Map each genotyped individual to the corresponding CNN, and train each CNN to obtain the classification accuracy on the validation dataset as its fitness value.
- Step 3: Use tournament selection to select the parent individuals based on the fitness, and then generate the new offspring achieving the same predefined size through the crossover and mutation operators.
- Step 4: Perform the environmental selection on the combined population to select a new population surviving into the next generation.
- Step 5: Go to step 3 if the termination condition is not satisfied. Otherwise, choose the individual with the best fitness and terminate the evolutionary process.

Indeed, the whole process of an EDL follows the procedure of an evolutionary algorithm, i.e., population initialization (step 1), fitness evaluation (step 2), offspring generation (step 3), and environmental selection (step 4). Generally, an evolutionary algorithm involves four parameters: the maximal generation number for the whole evolutionary process, the population size for the population initialization and the environmental selection, and the crossover rate as well as the mutation rate for the offspring generation. One major factor of the promising performance of evolutionary algorithms is the environmental selection to keep individuals with better fitness in the new population than the previous population. However, as described in step 2, the fitness of each individual in EDL is evaluated through the training of the corresponding CNN. For a better understanding of the fitness evaluation, the framework of training a CNN is provided in Algorithm 1.

Algorithm 1: Fitness Evaluation of a CNN in EDL

Input: The individual p, training dataset $\mathcal{D}_{train}$, validation dataset $\mathcal{D}_{valid}$, training epochs T, batch size b, learning rate γ, objective function $\mathcal{L}$.
Output: The fitness.

1 $net \leftarrow$ Decode p into the corresponding CNN architecture;
2 Add the classification layer to the end of *net*;
3 Randomly initialize the weights w in *net*;
4 $t \leftarrow 0$;
5 **while** $t < T$ **do**
6 **for** *each batch data in* $\mathcal{D}_{train}$ **do**
7 $\nabla w \leftarrow$ Compute the gradient by $\partial\mathcal{L}/\partial w$;
8 $w \leftarrow w - \gamma\nabla w$;
9 **end**
10 $t \leftarrow t + 1$;
11 **end**
12 Calculate the classification accuracy of *net* on $\mathcal{D}_{valid}$;
13 **Return** the fitness.

Generally, the training of a CNN is achieved by using SGD on a given training dataset through hundreds of epochs (steps 5–11). The objective of the training is to maximize the classification accuracy by adjusting millions of the weights (step 8) in CNN. For a moderate CNN on a commonly used benchmark dataset, e.g., CIFAR10 and CIFAR100 [15], the training process would take from several hours to days performing even on the high-performance GPUs. Since evolutionary algorithms belong to a class of population-based algorithms, their required numbers of evaluations are larger than that of SGD, which make the computational cost of EDL even higher. For example, on the CIFAR10 dataset, the large-scale evolutionary algorithm [12] consumed 11 days on 250 GPUs, and the hierarchical evolutionary algorithm [13] spent 1.5 days on 200 GPUs. Such a large number of GPUs may not be available to many researchers. To address such a challenge, it is of great importance to reduce the computational cost for EDLs by developing performance predictors, which could replace most of those expensive fitness evaluations, to speed up EDLs relying on limited computational resources.

2) AE-CNN: As the performance predictor is part of EDL in this paper, an EDL method should be provided before the performance predictor is detailed. In this paper, the AE-CNN algorithm developed by the authors is selected as the representative EDL, which is mainly based on the reasons that: 1) it shows promising performance among existing EDLs [48] and 2) the source code of AE-CNN is available to the public. Noting that the proposed performance predictor is applicable to any existing EDLs [8]–[14].

The AE-CNN algorithm [48] is an automatic EDL algorithm based on the building blocks of the state-of-the-art ResNet [49] and DenseNet [50], which has shown the promising performance on the CIFAR10 and CIFAR100 datasets [15] compared to state-of-the-art CNNs manually designed and other semi-automatic and automatic EDL algorithms [48]. The AE-CNN algorithm used in this paper is to prepare samples for training the random forest, and then the trained

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

random forest is used as the performance predictor in AE-CNN to verify the effectiveness and efficiency. In this section, we mainly introduce the genotype-to-phenotype mapping of AE-CNN for the completeness of the presentation, which enables the ability of evolutionary algorithms to model CNN architectures. These CNN architectures are the input to the random forest (the details of how to feed the architecture information to the random forest will be discussed in Section III).

The CNN architectures generated by AE-CNN are composed of the DenseNet Blocks (DBs), ResNet Blocks (RBs), and Pooling Blocks (PBs). Each DB or RB is composed of multiple DenseNet Units (DUs) and ResNet Units (RUs), respectively, while a PB consists of only one pooling layer. Each DU, RU, or PB differs in terms of the parameter settings. The parameters of a DU are the sizes of input and output (denoted by *in* and *out*, respectively), while those of an RU are the same as those of a DU, in addition to an increasing factor (denoted by *k*). The parameter of a PB is only the pooling type (i.e., the *maximal* or *mean* pooling type) because its other parameters are all set to fixed values. Because a DB/RB is composed of multiple DUs/RUs, the parameters of a DB/RB are the corresponding parameters of a DU/RU and the amount (denoted by *amount*) of DUs/RUs in a DB/RB. In addition, as the DBs, RBs, and PBs compose a CNN with an order, an extra parameter (denoted by *id*) is also used to represent the corresponding position in the CNN. Obviously, a CNN generated by AE-CNN is composed of sequential blocks which may be the DUs, RUs, or PBs. When the block is an RB, the parameters are the *id*, *amount*, *in*, and *out*; when the block is a DB, the parameters are *id*, *amount*, *k*, *in*, and *out*; when the block is a PB, the parameter is the pooling type denoted by *type*. Noting that the *in* of the current RB/DB should be equal to the *out* of its previous RB/DB for the reason of constructing a valid CNN.

3) Random Forest: Compared with other learning algorithms, such as neural networks, the random forest has the advantages of directly accepting the discrete data as input, almost having no extra parameters to tune and not relying on a large amount of training data [50]. These advantages are the exact reason for employing the random forest in this paper. Random forest [45] is an ensemble learning method by operating a set of decision trees for classification and regression tasks [46]. During the training of random forest, each decision tree randomly selects a part of the whole feature set and then learns the mapping from the selected features to the corresponding target. During the predicting process, each decision tree selects the same features as it selects in the training phase, and then output the corresponding prediction. The random forest uses the average output of the selected decision trees as its final output. Randomly selecting a part of the whole feature set is also the famous random subspace method [47], which has been shown to be simple yet effective.

B. Related Work

The existing performance predictors can be classified into two different categories: 1) performance predictors based on the learning curve and 2) E2EPPs, both of which are based on the training-predicting computational paradigm. The representatives of these two types are the FBO algorithm [29] and the Peephole algorithm [32].

The FBO algorithm uses a regression model based on Bayesian Optimization to predict the performance of a CNN. The regression model is built based on the data sampled from the learning curve regarding the first t epochs, i.e., a series of data pairs in different training epochs and the corresponding performance at these epochs. After the regression model that has been optimized by the Bayesian optimization method, it is used to predict the performance of the same CNN at the T-th epoch where $T > t$. The Peephole algorithm uses a number of CNN architectures and their corresponding performances that have been achieved by training the CNNs with T epochs, as the training samples to train a long-short time memory neural network [33]. The trained neural network directly predicts the performance of a new CNN-based its architecture, which is called the end-to-end mechanism because the end in the input is the raw data while the end in the output is the classification accuracy. Because the architecture cannot be directly used as the input data of the neural network, the Peephole algorithm employs the word vector technique to map the CNN architecture to a numerical value.

The major advantage of the FBO algorithm remains in the trained-CNN-free nature, i.e., it does not need any trained CNNs in advance. Because training a CNN is time consuming, varying from several days to weeks, the FBO algorithm is efficient. However, it will not be effective when the learning curve is not smooth because the curve fitting works under the assumption that the curve is smooth. In recent deep learning applications, the learning curve is usually not smooth because a schedule of learning rates is usually used. Once the learning rate is changed, the learning curve will have a nonsmooth segment. Another limitation of the FBO algorithm is the nonend-to-end nature (i.e., in predicting the performance of each CNN, a part of the training data regarding this CNN must be collected for training the predictor), which requires much more labor work when it is used. Owing to the end-to-end nature, the Peephole algorithm is more convenient for use. However, the major drawback of Peephole remains in the requirement of a large number of training samples, which results in added computational complexity of collecting train samples exceeding the EDL without using performance predictors. For example, Peephole used over 8000 fully trained CNN architectures as the training data. However, EDLs generally achieve promising performance by evaluating only hundreds of individuals. If we have enough computational resources to evaluate the 8000 CNN architectures, we will not need to develop the performance predictor. Such a limitation is largely caused by its adopted regression mode, i.e., the neural network-based algorithm, which typically highly relies on a large amount of labeled training data.

The proposed performance predictor has the advantages of the end-to-end manner and relying upon only limited training data, which address both limitations of the existing performance predictors discussed above.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

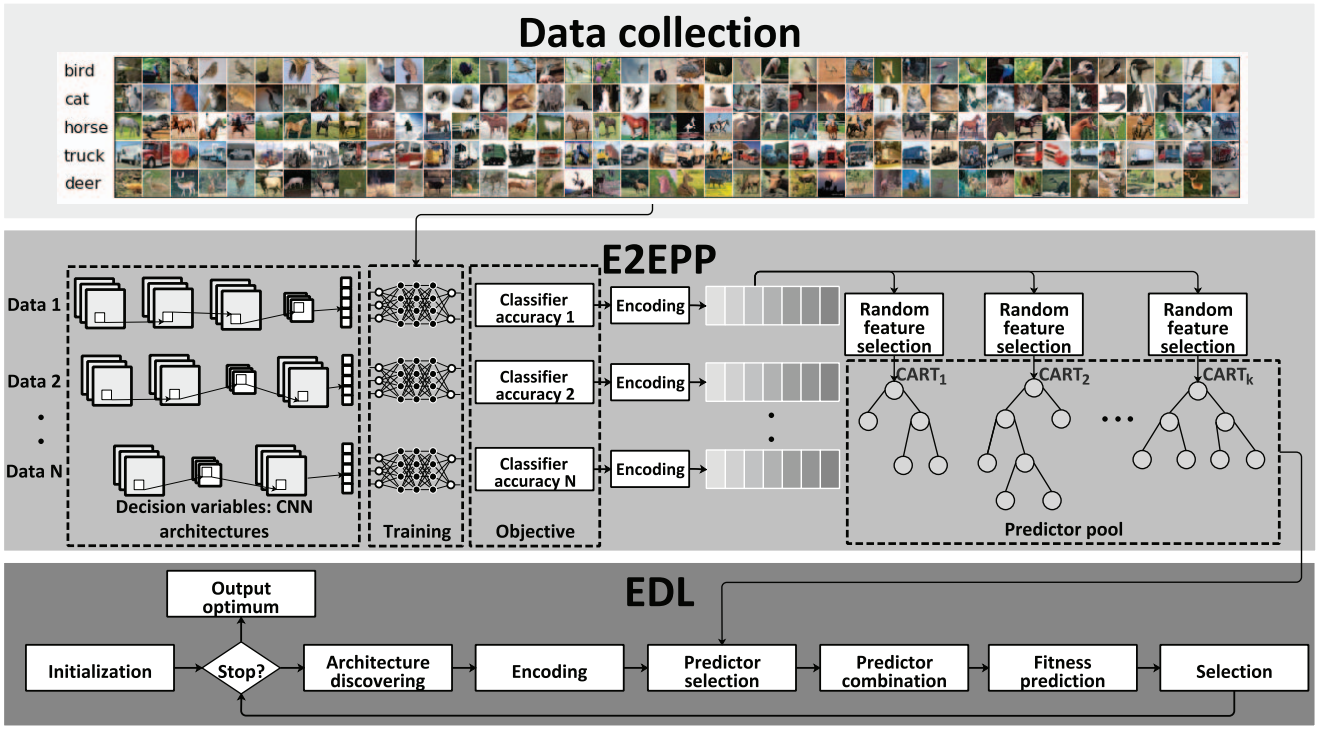


Fig. 1. Main frame of the proposed algorithm.

III. Proposed Algorithm

As mentioned, the main challenge of EDL is the high computational cost of evaluating a single CNN architecture. Inspired by offline SAEAs, we use random forest [46] as the fitness predictor to replace the expensive fitness evaluation. For a better understanding, Fig. 1 shows the framework of the proposed performance predictor as well as the associated EDL. The random forest-based performance predictor is learned from a number of different CNN architectures that have been trained for a classification task with their accuracy. Compared with the CNN training process, the random forest construction and prediction is computationally cheaper and can be repeatedly used throughout the evolutionary optimization. Thus, the high computational cost in EDL can be relieved.

As shown in Fig. 1 that is composed of three blocks, i.e., the data collection, E2EPP, and EDL. The proposed E2EPP performance predictor is part of EDL. First, a set of training data is collected for training the random forest-based predictor, where the collection is achieved by performing the corresponding EDL without using E2EPP. Each data sample is composed of the CNN architecture and the corresponding classification accuracy that is obtained by training the CNN from scratch. Second, those architectures are encoded into discrete code (shown in Section III-A) for building the random forest-based predictor pool with a large number (say K) regression trees (denoted as CARTs) [51] (shown in Section III-B). During each generation of the EDL, the newly generated CNN architecture is encoded as the input to the random forest, and then its performance is predicted by using the adaptive combination of CARTs from the predictor pool (shown in Section III-C). When the EDL terminates, the CNN architecture that has the best prediction performance is output. Noting that there is no further CNN training during the optimization process.

Since the CNN architecture is presented as discrete variables, random forest [52], which is suited for discrete regression tasks and also relies on only limited labeled data [53], is adopted as the fitness predictor in the proposed algorithm. Also, as a selective ensemble of CARTs considering both global and local landscape, the random forest-based predictor is adaptively updated over the generations of the evolutionary search, where the population is distributed in different local regions and a fixed random forest cannot guarantee accuracy in those local regions. In the following sections, we will first introduce the details of encoding of the CNN architectures to the proper format for the random forest, training of the random forest, and fitness prediction using the trained random forest in Sections III-A–III-C, respectively. Then, the strength and the weakness of the proposed performance predictor are summarized in Section III-D.

A. Encoding

The encoding operates on the trained CNNs whose architectures are randomly initialized. To collect this data, we use AE-CNN to randomly generate a set of valid CNN architectures. A valid CNN architecture means that this architecture can be trained with a predefined training routine without any exceptions such as the out-of-memory errors causing a zero classification accuracy. Based on the description shown in Section II-A2, the collected training data are summarized as below.

1) *RBs and DBs:* Each generated CNN architecture is composed of four RBs and four DBs at most, the number of

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Encoding a CNN Architecture

Input: The CNN architecture $\mathcal{A}$, the maximal number B_n of DBs and RBs, the maximal number N_p of PBs.
Output: The encoded architecture information.

```
b_list ← ∅;
p_list ← ∅;
l ← Calculate the number of blocks in A;
for i ← 1 to l do
    block ← Get the i-th block of A;
    if block is a RB then
        Put 1 into b_list;
        Put the values of the out and amount of block into b_list;
    else if block is a DB then
        Put the values of the k, out and amount of block into b_list;
    else
        if block is a maximal pooling layer then
            Put 1 to p_list;
        else
            Put 0 to p_list;
        end
        Put i to p_list;
    end
end
Put zero to b_list until |b_list| = 3N_b;
Put zero to p_list until |p_list| = 2N_p;
Return b_list ∪ p_list.
```

Algorithm 3: Performance Predictor Training

Input: The K CARTs, the encoded training data $\mathcal{D}_{train}$, the feature number m.
Output: The K trained CARTs and their selected feature ids.

```
I ← ∅;
for i ← 1 to K do
    CART ← Select the i-th CART from CARTs;
    v ∈ R^m ← Randomly generated a vector from [0, 1];
    I_i ← Collect the position of the elements whose values are greater than 0.5 in v;
    Train CART on the features whose ids are in I_i;
    I ← I ∪ I_i;
end
```

Output: The K trained CARTs and the corresponding selected feature ids I.

output channels of each block varies between $[32, 512]$ that is set based on the conventions of state-of-the-art CNNs [50], [54].

2) *PBs:* Each generated CNN architecture contains four pooling layers at most. There are two types of PBs: a) MAX and b) MEAN.

Generally, we encode a CNN architecture into a chromosome with $3N_b + 2N_p$ discrete variables, when the maximal number of RBs and DBs is N_b and the maximal number of PBs is N_p. For the first $3N_b$ variables, each RB or DB is encoded into a triplet as [*type*, *out*, *amount*], where the block type for RBs is set to 1, and that for DBs are set to 12, 20, and 40 when k is equal to 12, 20, and 40, respectively. Noting that the parameter of *in* for each RB/DB is not encoded because it can be calculated by the *out* of its previous RB/DB, and the smaller number of decision variables could result in a better performance for regression model when the training data is limited [46]. For the following $2N_p$ variables, each pooling layer is encoded into a pair as [pooling type, layer position], the *maximal* and *mean* pooling types are presented by 1 and 0, respectively. If a CNN architecture has b RBs and DBs, and p PBs, then its $3b + 1$th to $3N_b$th variables are set to zeros, and its $3N_b + 2p + 1$th to $3N_b + 2N_p$th variables are set to zeros as well. Therefore, the performance predictor by using random forest is based on the input data with $3N_b + 2N_p$ discrete decision variables, and the output is a continuous value within the range of $[0, 1]$. Algorithm 2 shows the details of encoding a CNN architecture into the data that can be directly used by the random forest, and $|\cdot|$ is a countable operator.

B. Training of the Random Forest

A large number of CARTs are generated in the predictor pool. Each CART is trained by the whole training data with a random subset of features (i.e., discrete variables), where each discrete variable is assigned a probability of 0.5 in order to maximize the diversity of the predictor pool [55]. Each node of a CART presents a rectangle region in the decision space. The mean squared error of the output of those samples in that region (node) determines whether this node needs splitting or not (i.e., whether the mean squared error decrease is smaller than a set threshold T_s [51], and if so, this node is a leaf node). When K CARTs are obtained, the predictor pool is ready for the optimizer. The details of training the CARTs are shown in Algorithm 3.

C. Performance Prediction

Since the computational cost of training a single CNN is very high, it is impractical to obtain any new training data for the predictor pool during the optimization process. Thus, those predictors cannot be updated or validated, which is the main challenge of offline SAEAs [40]. To deal with the lack of training data, a large number of surrogate models are employed as ensemble members in a recent offline SAEA [44]. The results indicate that a selective ensemble surrogate can effectively improve the robustness of the obtained solution when the new training data is unavailable. Also, those ensemble members are adaptively combined in each generation to provide local information on the current population. Motivated by the combination strategy that was originally proposed in our previous work [44] for addressing decision variables having continuous values, we employ it to update the random forest-based predictor in every generation in the proposed algorithm, i.e., we choose Q CARTs from the predictor pool and then use their average prediction as the fitness value. Although the decision variables of the proposed algorithm are discrete, it is

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

experimentally found that the combination strategy still works well.

In each generation, all of the K trained CARTs re-estimated the performance on the CNN $\mathcal{A}^b$ that has the best-predicted fitness value; and then Q out of the K CARTs are uniformly selected from the K ordered CARTs based on their prediction values on $\mathcal{A}^b$. The Q CARTs are combined as the ensemble performance predictor to evaluate both parent and offspring population. Such selection is based on the performance diversity of CARTs around the current best CNN architecture $\mathcal{A}^b$. After that, the generated CNN architectures $\mathcal{A}$ are evaluated by using the ensemble predictor of Q CARTs. Thus, the adaptive predictor can balance the global tendency and local information in the fitness landscape, where the combination of K CARTs predicts the global average landscape and that of Q diverse CARTs in a small area refines the local landscape. The details of the prediction process in a generation are shown in Algorithm 4.

Algorithm 4: Performance Predicting

Input: The K trained CARTs, the selected features ids I of each CART, the current best CNN $\mathcal{A}^b$, the number of most diverse prediction Q, the generated architectures $\mathcal{A}$ to be evaluated.
Output: The fitness values of $\mathcal{A}$.

```
Y ← ∅;
A^b_encoded ← Encode A^b based on the details shown in Section III-A;
for i ← 1 to K do
    CART ← Select the i-th CART from CARTs;
    x ← From A^b_encoded select the elements whose ids are in I_i;
    y ← Use CART predict the classification accuracy on x;
    Y ← Y ∪ x;
end
Y ← Order the elements in Y;
I^CART_selected ← Uniformly select Q CARTs based on Y;
for i ← 1 to |A| do
    F_i ←the mean prediction of Q selected CARTs (I^CART_selected) on |A_i|;
end
Return F.
```

D. Strength and Weakness of E2EPP

As have been introduced in Section II-B, the limitations of the existing performance predictors are the nonend-to-end nature, the strict assumption on the smoothness of the learning curve, and the availability of the large training samples. The proposed method has been carefully designed to address these limitations.

The proposed algorithm is end-to-end and does not rely on the learning curve no matter whether it is smooth or not. First, the end-to-end nature is more convenient in practice because we do not need to prepare the training data in predicting the performance of each CNN. Second, because the proposed algorithm does not need to fit the learning curve, the predicted performance is better than the existing approaches based on the learning curve. This is theoretically evidenced by the universal approximation theorem [56] that the learning curve-based approaches can achieve promising performance only when the learning curve is smooth. However, in practice, the learning curve is not always smooth.

On the other hand, the proposed algorithm does not require a large number of training samples. Most existing approaches employing deep learning techniques succeed subject to the availability of a large amount of training data. In the CNN performance prediction, such training data is collected by training a lot of CNNs from scratch. However, the training is time consuming even if performing on GPUs, while the main goal of designing performance predictors is to save the time of training CNNs. If we put enough time to collect a sufficient amount of training data, the performance predictor design will lose its original purpose. In the proposed method, we use the random forest as the base operator to learn the mapping between the CNN architecture and its performance. The reason is that random forest can achieve a promising performance on limited amount of training data, which has been theoretically proven and practically investigated [45], [46].

Unfortunately, the weakness of the proposed algorithm is the unknown of the minimal number of training samples for achieving promising performance, while the minimal number is case by case for different tasks. In practice, we need to use an incremental strategy to sample training samples until the desired performance is achieved.

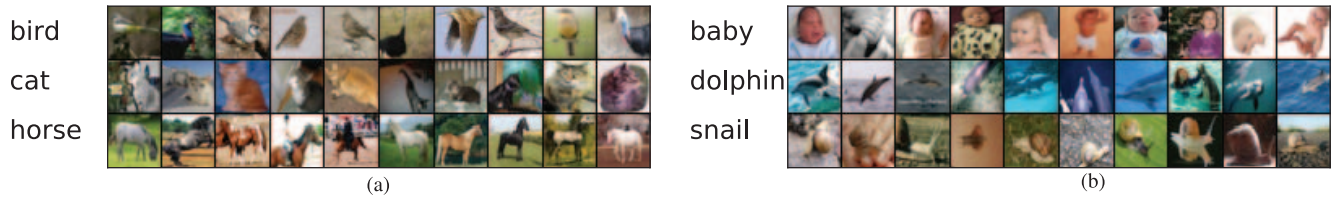
IV. Experiment Design

In order to verify the effectiveness and efficiency of the proposed performance predictor, a series of experiments are carefully designed and performed. Although the proposed performance predictor aims at speeding up the fitness evaluation of EDL, the ultimate goal of the performance predictor is to find the best CNN architecture that achieves a promising classification performance on the image data at hand. Therefore, two experiments are performed in this paper: 1) investigating the classification performance of the proposed performance predictor with AE-CNN and 2) inspecting the efficiency of the proposed performance predictor. In this section, the selected peer competitors and benchmark datasets, as well as the parameter settings for these two types of experiments, are detailed.

A. Peer Competitors

In comparing the classification performance, the chosen peer competitors are divided into three different categories: 1) the state-of-the-art CNNs whose architectures are manually designed; 2) the state-of-the-art CNN architecture designs based on nonevolutionary algorithms (mainly based on reinforcement learning); and 3) the state-of-the-art EDL algorithms. The first category covers DenseNet [50], ResNet [54], Maxout [57], VGG [58], network in network [59], highway network [60], all-CNN [61], and FractaNet [62]. Considering

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 2. Examples of CIFAR10 and CIFAR100 shown in (a) and (b), respectively. Each row represents two samples from the same category and the words in the left refer to the corresponding category name.

the promising performance of ResNet, we use its two different versions: ResNet with the depths of 101 and 1202, respectively. For the convenience of the discussion, they are denoted as ResNet (depth = 101) and ResNet (depth = 1202), respectively. The second category consists of NAS [9], MetaQNN [8], EAS [9], and Block-QNN-S [10]. The third category includes genetic CNN [11], large-scale evolution [12], hierarchical evolution [13], and CGP-CNN [14]. Considering the proposed performance predictor is introduced in a case study of AE-CNN, AE-CNN combined with E2EPP (denoted as AE-CNN+E2EPP) is chosen to perform this experiment.

For the second experiment, the selected peer competitors are the existing performance predictors [30], [32] discussed in Section II-B.

B. Benchmark Datasets

Benchmark datasets are only required by the first experiment. The CIFAR10 and CIFAR100 datasets are selected as the benchmark datasets. Both benchmark datasets are selected mainly because of their wide adoption by the state-of-the-art CNNs and CNN architecture design algorithms.

CIFAR10 is a 10-category natural object classification dataset, consisting of a training dataset with 50 000 images and a test dataset with 10 000 images, and each image has the dimension of 32×32. In the training dataset, each category has roughly the equal number of samples, while in the test dataset, each category has the exact same number of samples.

CIFAR100 is just like CIFAR10, except that it is 100-category. Because CIFAR100 has the same number of images in the training images and test images as those of CIFAR10, each category in CIFAR100 only has one-tenth of images as that in CIFAR10. Due to the smaller number of training data in each category and a larger number of classification categories than CIFAR10, CIFAR100 is a more complex dataset for classification than CIFAR10.

For each image of both benchmark datasets, the object is to be classified commonly occupies a small area of the entire image, and also the size, the area, and the position of each object differ to each other in other images, even when they are from the same category. An example of these two benchmark datasets are shown in Fig. 2 for a glance, where each row denotes the objects from the same category, and the label leading in each row denotes the ground truth of the corresponding object.

C. Parameter Settings

In this section, the parameter settings for generating the training data from AE-CNN are detailed at first, and then those of the proposed performance predictor are introduced next. Noting that these parameter settings are applied to both experiments.

Based on the conventions of the deep learning community, the SGD algorithm is used to train the CNN architectures whose weights are initialized with the commonly used Xavier method [63]; the batch size is set to 128; the weight decay is set to 5×10^{-4}; each CNN is trained 350 epochs, and the learning rate is set to 0.01 for the first epoch and the 151st to the 249th epoch, 0.1 for the second to the 150th epoch, and 0.001 for the remaining epochs. The classification accuracy is evaluated on the "validation dataset."[1] Because the used benchmark datasets do not have the corresponding validation dataset, 20% images are randomly selected from the corresponding training dataset as the validation dataset based on the conventions. All the CNNs are trained on three GPUs with the same model of NVIDIA 1080TI. In addition, we set both maximal numbers of RB and DB to 4 because of the limited GPU memory. Because each pooling operation halves the input size one time, and the input sizes of CIFAR10 and CIFAR100 are both 32×32, the maximal number of pooling layers is set to 4. Furthermore, based on the conventions of DenseNet, k is selected from $\{12, 20, 40\}$, and the maximal numbers of DUs in a DB are 10 when $k = 12$ and $k = 20$, and 5 when $k = 40$. The maximal number of RUs in an RB is set to 10. Based on our configuration for generating the training data, N_b and N_p are set equal to 8 and 4, respectively, and the decision variable length of each training data for the random forest is 32 ($8 \times 3 + 4 \times 2$). To build the performance predictor pool, we generate 1000 CARTs. The threshold of stopping node splitting is set as $1e{-}4 \times \sigma^2$ (σ^2 is the variance of the training data) when growing each CART. In each generation, 100 CARTs are selected to combine the random forest-based predictor. All the parameter settings are summarized in Table I.

In performing the CNN architecture design by using AE-CNN and the proposed E2EPP method, we set both the maximal generation number and the population size to 20, the probabilities for the crossover and mutation are set to 0.9 and 0.1, respectively, as suggested in AE-CNN. Noting that before E2EPP is used for each generated CNN architecture, the CNN architecture is evaluated in the GPUs for one epoch so that the

[1]We are aware of different definitions on the validation (data)set from literature. Based on the convention of the machine learning community, the validation set is used only for controlling overfitting and model selection, and the selected model cannot be updated and improved via the validation set. In EDL, after the individuals (models) are selected based on their performance of the validation set, they are updated and possibly improved by genetic operators during evolution. So we use the quotation marks to highlight its different meaning here from its traditional meaning.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF THE PARAMETER SETTINGS

Parameter name	Parameter value
batch size	128
weight decay	5×10^{-4}
training epochs	350
learning rate	0.01 for 1, and 151–249 epochs; 0.1 for 2–150 epochs; 0.001 for 251–350 epochs
k of the DB	$\{12, 20, 40\}$
maximal number of DUs in a DB	10 when $k = 12$ and $k = 20$; 5 when $k = 40$
maximal number of RUs in a RB	10
maximal number of DBs (N_p)	4
maximal number of RBs (N_b)	8
generation number	20
population size	20
crossover probability	0.9
mutation probability	0.1
number of CARTs	1000
threshold of stopping node splitting	$1e-4 \times \rho^2$ (ρ^2 is the variance of the training data)
number of selected CARTs	100

generated CNN can be normally evaluated by E2EPP, i.e., the employed GPUs can carry the CNN architecture and will not lead to an out-of-memory error. This is also for keeping a similar data distribution between the training data and the test data for E2EPP. After the evolutionary process, the individual with the best classification accuracy is chosen and then fully trained with the same training routine as collecting the training data for five independent runs, and the best result is reported, which is followed the conventions from deep learning community.

V. EXPERIMENTAL RESULTS

The results of the designed experiments are presented and analyzed in this section. Specifically, the classification performance of AE-CNN+E2EPP, in terms of the classification accuracy and the consumed "GPU days," are elaborated in Section V-A. In order to extensively investigate the proposed E2EPP performance predictor, its efficiency and effectiveness are individually tested, and the corresponding experimental results are shown in Sections V-B and V-C, respectively. In addition, the comparision to radial basis network (RBN) has also been done to show the promising performnace of random forest employed in the proposed E2EPP performance predictor.

A. Overall Results

Table II presents the experimental results in terms of the classification accuracy and consumed GPU days of the compared algorithms. Specifically, Table II is divided into five rows: the second to fourth rows denote the state-of-the-art CNNs manually designed, the state-of-the-art CNN architecture design algorithms based on nonevolutionary algorithms, and the state-of-the-art CNN architecture design algorithms based on evolutionary algorithms (i.e., the EDL algorithms), respectively; the last row shows the result of the proposed performance predictor used in AE-CNN (i.e., AE-CNN+E2EPP). In addition, the first column shows the names of the compared algorithms; the second and the third columns show the classification accuracy on CIFAR10 and CIFAR100, and the fourth column shows the consumed GPU days for achieving the corresponding classification accuracy by the corresponding architecture design algorithms. These architecture design algorithms generally report their consumed numbers of GPUs and days. For the convenience of the comparison, we unify them by using GPU days as an indicator expressing the utilized computational resource. For example, one GPU day means a GPU is fully consumed in one day to achieve the classification accuracy. All the results of the peer competitors in this table are extracted from their seminal papers, in addition to the GPU days that we convert it by multiplying the number of GPUs with the number of days taken. The symbol "–" means the corresponding paper does not provide the corresponding result publicly available. Noting that the GPU days of AE-CNN+E2EPP is mainly caused by collecting the training data but not performing E2EPP.

For the classification accuracy obtained by the state-of-the-art CNN manually designed, on the CIFAR10 benchmark dataset, AE-CNN+E2EPP achieves, on the average, the classification accuracy of 1.5% higher than ResNet (depth = 101), VGG and All-CNN, 2.04% higher than ResNet (depth = 1202) and Highway Network, and even 3.78% higher than Maxout and Network in Network, respectively, while slightly lower than DesnNet (0.04%) and FractalNet (0.06%). On the CIFAR100 benchmark dataset, AE-CNN+E2EPP, whose classification accuracy is 77.98%, outperforms all other peer competitors that achieve the classification accuracy varying from 77.70% (by FractalNet) to 61.40% (by Maxout). AE-CNN+E2EPP demonstrates the promising performance among peer competitors in this category.

Compared with nonevolutionary algorithms, AE-CNN+E2EPP achieves the better classification accuracy than NAS and MetaQQ, but a little worse than EAS and Block-QNN-S on CIFAR10; on CIFAR100, the classification accuracy of AE-CNN+E2EPP is significantly better than that of MetaQQ while slightly worse than that of Block-QNN-S. In addition, NAS, MetaQNN and Block-QNN-S achieve their classification accuracies by consuming 22 400, 100, and 90 GPU days, respectively, while AE-CNN+E2EPP only consumes 8.5 GPU days. In addition, AE-CNN+E2EPP also consumes fewer GPU days than that of EAS.

Compared with other EDLs, AE-CNN+E2EPP shows lower classification accuracy than that of hierarchical evolution, while higher than those of genetic CNN, large-scale evolution, and CGP-CNN on CIFAR10. On CIFAR100, AE-CNN+E2EPP shows the highest classification accuracy than others. Furthermore, AE-CNN+E2EPP also consumes the fewest GPU days than other peer competitors in this category.

In fact, EAS, Block-QNN-S, hierarchical evolution, and CGP-CNN are all semi-automatic CNN architecture design algorithms, i.e., they design the CNN architectures based on both the expertise of manually designed CNNs and the automatic ways of EDLs. To this end, it is reasonable that CNN+E2EPP is inferior to them in terms of the classification accuracy. In summary, AE-CNN+E2EPP wins 26 times out of the 30 classification accuracy comparisons, and also consumes the least GPU days among all architecture design peer competitors.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF AE-CNN+E2EPP AND THE PEER COMPETITORS IN TERMS OF THE CLASSIFICATION ACCURACY (%) AND THE CONSUMED GPU DAYS ON THE CIFAR10 AND CIFAR100 BENCHMARK DATASETS

	Peer Competitors	CIFAR10	CIFAR100	GPU Days
state-of-the-art CNNs manually designed	DenseNet [50]	94.76	75.58	–
	ResNet (depth=101) [54]	93.57	74.84	–
	ResNet (depth=1,202) [54]	92.07	72.18	–
	Maxout [57]	90.70	61.40	–
	VGG [58]	93.34	7.95	–
	Network in Network [59]	91.19	64.32	–
	Highway Network [60]	92.28	67.61	–
	All-CNN [61]	92.75	66.29	–
	FractalNet [62]	94.78	77.70	–
CNN architecture design algorithms based on non-evolutionary methods	NAS [9]	93.91	–	22,400
	MetaQNN [8]	93.08	27.14	100
	EAS [9]	95.77	–	10
	Block-QNN-S [10]	95.62	79.35	90
CNN architecture design algorithms based on evolutionary methods	Genetic CNN [11]	92.90	70.95	17
	Large-scale Evolution [12]	94.60	77.00	2,750
	Hierarchical Evolution [13]	96.37	–	300
	CGP-CNN [14]	94.02	–	27
	AE-CNN + E2EPP	94.70	77.98	8.5

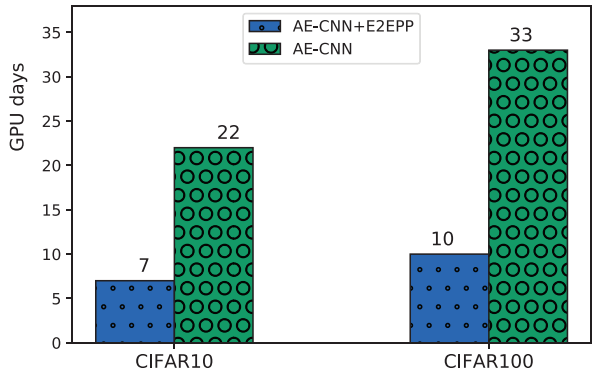


Fig. 3. Consumed GPU days of AE-CNN and AE-CNN+E2EPP when achieving the same classification accuracy on CIFAR10 and CIFAR100.

B. Efficiency of E2EPP

The experimental results presented in Table II have indicated that AE-CNN can achieve promising classification accuracy when the proposed E2EPP is used, which is the ultimate goal of designing the performance predictor. In this section, we will specifically investigate the efficiency of E2EPP by disabling and enabling it in AE-CNN. To do a fair comparison, AE-CNN is performed with the same parameter settings as those of AE-CNN+E2EPP on both CIFAR10 and CIFAR100. We record the consumed GPU days of AE-CNN when it has a new generated CNN architecture whose fitness value satisfies the condition formulated by

$$\left|\frac{f_1 - f_2}{f_2}\right| \leq 0.01 \tag{2}$$

where f_1 denotes the fitness of the new CNN architecture generated by AE-CNN, and f_2 denotes the fitness of AE-CNN+E2EPP shown in Table II.

Fig. 3 shows the comparisons of the consumed GPU days when AE-CNN satisfies the condition of (2) on CIFAR10 and CIFAR100. The numbers above each bar display the consumed GPU days by the corresponding algorithm. The bars filled with the symbol of "·" denote the results of AE-CNN+E2EPP, while the bars filled with the symbol of "∘" denote that of AE-CNN. Clearly, it can be seen from Fig. 3 that E2EPP has significantly improved the efficiency of AE-CNN for finding the best CNN architectures. Particularly, E2EPP saves 214% and 230% fewer GPU days on CIFAR10 and CIFAR100, respectively, when it is used in AE-CNN. Therefore, the goal of designing such a proposed performance predictor, in terms of speeding up the fitness evaluation, has been achieved.

TABLE III
MSE, KTAU, COD AND THE USED TIME OF FBO, PEEPHOLE, AND E2EPP ALGORITHMS ON THE **CIFAR10** DATASET

	MSE	KTau	CoD	Time (seconds)
FBO	0.0077	0.2170	-4.346	3.702
Peephole	0.0019	0.5324	0.3765	359.296
E2EPP	0.0006	0.6604	0.5184	3.813

C. Effectiveness of E2EPP

In order to check the effectiveness of E2EPP, the FBO [30] and Peephole [32] algorithms, which are the representatives of existing performance predictors as detailed in Section II-B, are selected to do the quantitative and qualitative comparisons against E2EPP. For the quantitative comparison, we choose the mean square error (MSE), Kendall's Tau (KTau) [64], and the coefficient of determination (CoD) [65] as the indicators suggested in [32]. Specifically, KTau measures the correlation between the ranks of the prediction and their true ranks. A higher KTau value implies the higher correlation and the KTau value varies between $[-1, 1]$. The CoD measures the closeness degree of the prediction to its true value, and its value ranges from $-\infty$ to 1 where a larger value means the closer degree. For the qualitative comparison, we scatter the true values with the corresponding prediction on a 2-D space where the horizontal axis denotes the true values while the vertical axis denotes the prediction. The parameter settings of FBO and Peephole are set based on the suggestions in [32]. In addition, the experiment on each benchmark dataset is performed ten times.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

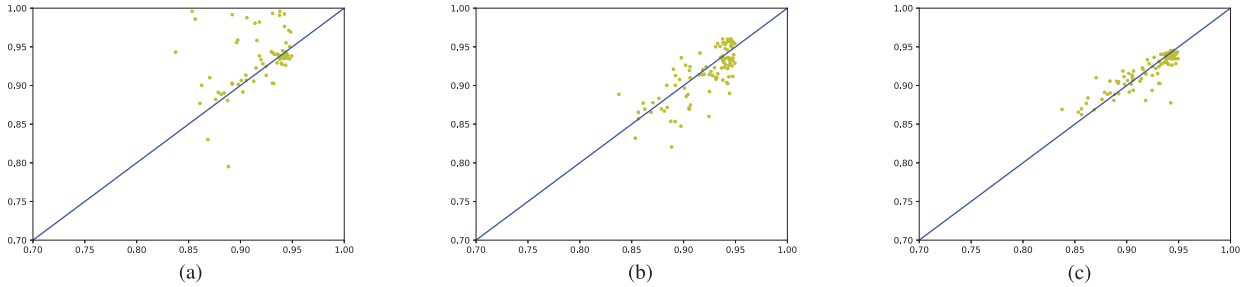


Fig. 4. Prediction and the true values of (a) FBO, (b) Peephole, and (c) E2EPP on the CIFAR10 dataset. In each subfigure, the horizon axis denotes the true values, while the vertical axis denotes the corresponding prediction.

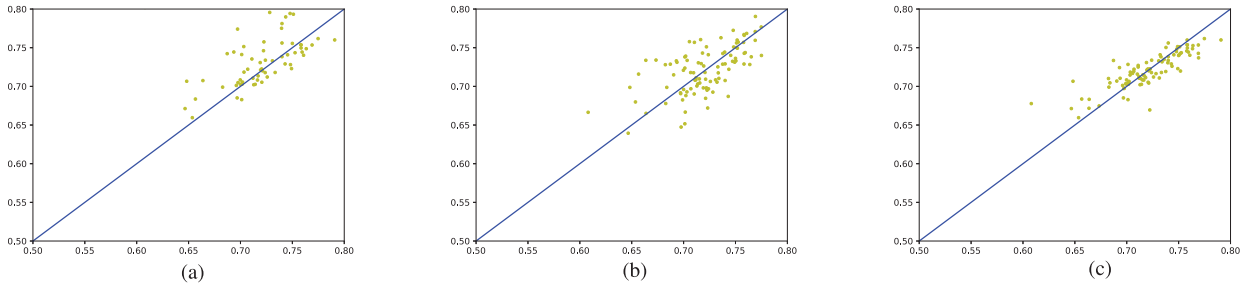


Fig. 5. Prediction and the true values of (a) FBO, (b) Peephole, and (c) E2EPP on the CIFAR100 dataset. In each subfigure, the horizon axis denotes the true values, while the vertical axis denotes the corresponding prediction.

TABLE IV
MSE, KTAU, COD AND THE USED TIME OF FBO, PEEPHOLE, AND E2EPP ALGORITHMS ON THE **CIFAR100** DATASET

	MSE	KTau	CoD	Time (seconds)
FBO	0.0078	0.1923	-2.631	3.725
Peephole	0.0024	0.5412	0.3238	458.652
E2EPP	0.0015	0.6501	0.3872	3.514

The quantitative comparison results on CIFAR10 and CIFAR100 are shown in Tables III and IV, respectively. In addition, we also add the consumed time (seconds) measuring how long E2EPP, FBO, and Peephole take to finish the experiment. Clearly as can be seen from Table III, FBO and Peephole obtain the MSE of 0.0077, and 0.0019, respectively, while E2EPP obtains the MSE of 0.0006 that is an order of magnitude less than FBO and Peephole can. This means that E2EPP could achieve a better prediction than FBO and Peephole. For the KTau indicator, FBO, Peephole and E2EPP achieve the values of 0.2170, 0.5324, and 0.6604, respectively, which imply that the performance predicted by E2EPP has the best correlation. For the CoD indicator, E2EPP again wins the highest value of 0.5184 (-4.346 for FBO and 0.3765 for Peephole). Based on the definition of CoD, the predicted value of E2EPP is the closest one to the ground truth value among the comparisons. In addition, E2EPP uses 3.813 s for the prediction, which is a little worse than FBO that uses 3.702 s. However, it is still much better than Peephole that uses 359.296 s. The similar comparison results can also be investigated from Table IV. Specifically, E2EPP obtains the MSE of 0.0015 that is about half of that from Peephole and 1/5 of that from FBO, i.e., E2EPP has the minimal prediction error than the others. Furthermore, FBO obtains the KTau indicator value of 0.1923, while Peephole and E2EPP obtain the values of 0.5412 and 0.6501, respectively. For the CoD indicator, E2EPP achieves the best one, i.e., 0.3872, while FBO and Peephole acquire -2.631 and 0.3238, respectively. This shows the best prediction performance of E2EPP on the CIFAR100 dataset. In addition, E2EPP consumes 3.514 s to finish the prediction, which is the best among FBO (3.725 s) and Peephole (458.652 s).

The qualitative comparisons on CIFAR10 and CIFAR100 are scattered in Figs. 4 and 5, respectively, where the x-axis of each point [denoted as (x, y)] represents the true value and y-axis denotes the corresponding prediction. For the convenience of the comparison, we also plot the line $y = x$ in each figure. The points standing above $y = x$ mean the predictions are higher than the true values, while the points standing below $y = x$ means the predictions are lower than the true values. The more points closer to $y = x$, the better prediction of the corresponding performance predictor. As can be clearly seen from Fig. 4(a) and (b), FBO has only half of the total points that are close to the line of $y = x$, and Peephole achieves better results by having more points close to the line of $y = x$. However, it is clearly observed from Fig. 4(c) showing the results of E2EPP, almost all of the points are close to the line $y = x$. This means that the predicted values of E2EPP have minimal prediction errors, which is consistent with the quantitative analysis shown in Table III. From Fig. 5(a)–(c), that shows the results performed on CIFAR100 by FBO, Peephole and E2EPP, respectively, it can be seen that FBO and Peephole achieve the similar results that have a wide spread of points around the line of $y = x$, while the results of E2EPP have the closer distance to the line $y = x$. This also shows the same conclusion from the quantitative analysis observed in Table IV.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF AE-CNN+E2EPP AND AE-CNN+RBN IN TERMS OF THE CLASSIFICATION ACCURACY (%) ON THE CIFAR10 AND CIFAR100 BENCHMARK DATASETS

	CIFAR10	CIFAR100
AE-CNN + E2EPP	94.70	77.98
AE-CNN + RBN	82.33	70.20

D. Comparison to Radial Basis Network

In order to demonstrate the promising performance of the random forest employed by the proposed E2EPP performance predictor, the comparison to RBN is performed, and the results are shown and analyzed in this section.

Specifically, an RBN is a three-layer neural network, except that the activation function of each unit in the hidden layer is a Gaussian function, and the output layer is a linear weight summation of the output of the hidden layer. The RBN is often used for regression tasks, which is similar to the function of random forest used in the proposed E2EPP performance predictor. Another reason for choosing RBN here for the comparison is that RBN is the base of our previous work [44] that motivates the design of E2EPP in this paper. For a fair comparison, we only replace the random forest by an RBN, while the other parts are kept the same as that used for the experiment shown in Section V-A. For the convenience of the discussions, we denote the RBN used by the AE-CNN algorithm as AE-CNN+RBN.

In the comparison, we only show the classification accuracies on CIFAR10 and CIFAR100, but not showing the consumed GPU days since the performance of both AE-CNN+E2EPP and AE-CNN+RBN does not rely on GPUs. The number of GPU days shown in Table II of AE-CNN+E2EPP is used for measuring the validity of the generated CNN architectures during the evolutionary process. In addition, the number of hidden units of the RBN is set to the same as the input size based on the conventions of practice [44]. The comparison results are shown in Table V.

It can be seen from Table V that AE-CNN+E2EPP achieves the classification accuracies of 94.70% and 77.98% on CIFAR10 and CIFAR100, respectively, while AE-CNN+RBN achieves those of 82.33% and 70.20%, respectively. This implies that the random forest employed in the proposed E2EPP performance predictor is better than RBN. In fact, it is easy to find the proof of this conclusion, when the RBN is used, the mean and standard deviations of the Gaussian functions are calculated based on the training data. However, the training data in E2EPP is encoded with some numerical numbers that do not refer to the real meaning. In addition, the RBN has been well-known to be suitable for continuous data, while the training data in E2EPP is discrete. In summary, AE-CNN+E2EPP outperforms AE-CNN+RBN on the CIFAR10 and CIFAR100 benchmark datasets.

As can be seen above, we have performed four groups of experiments that are carefully designed to verify the promising performance of the proposed E2EPP performance predictor. Particularly, the first is for investigating whether E2EPP could enable EDL to achieve the competitive performance in designing the CNN architectures. The second validates whether E2EPP could accelerate EDL or not. The third quantitatively and qualitatively investigates whether E2EPP could outperform the existing approaches regarding the performance prediction, and the fourth investigates whether the random forest employed in E2EPP could perform better than the RBN that is another commonly used regression model. In addition to the third one, all the others are performed in the case of AE-CNN. All the experimental results demonstrate the effectiveness and efficiency of the proposed E2EPP performance predictor. From these experimental results, we can recognize that the performance predictor is indeed able to speed up EDL, and the end-to-end nature is convenient to use in practice by avoiding manual interventions during the performing of EDL. In addition, the random forest, as an ensemble approach, achieves the promising performance but relying on only limited training samples, which is a good characteristic especially in EDL where sufficient training data is not easy to obtain. Furthermore, the random forest could directly use the discrete number as its input, which is also another good characteristic in predicting the performance of CNNs when the architecture information is used as the input data. This implies that the ensemble methods depend on discrete input data may be a proper choice in designing the performance predictors, which is not only suitable to EDL but also other deep learning model selection methods.

VI. CONCLUSION

The goal of this paper was to develop an effective and efficient end-to-end performance predictor, named E2EPP, to speed up EDLs in designing the best CNN architectures for given tasks. This goal has been achieved by proposing a random forest-based offline surrogate model. In order to enable the random forest to accept the CNN architectures as its input data, we also develop an encoding schema that is able to uniquely map the description of a CNN architecture into a numerical decision variable. In addition, we took the advantage of random forest in terms of the ensemble characteristic that each tree in the random forest randomly selects a subspace of the whole features as its training data. Furthermore, in order to improve the diversity of used off-line model, only a part of the decision trees in the random forest is selected based on their prediction on the best individual to do the prediction, which in turn enhances the accuracy of the predicted values. The proposed performance predictor is first examined by integrating it into an existing EDL algorithm, which is called AE-CNN+E2EPP, on CIFAR10 and CIFAR100. Through the comparisons with eight CNNs whose architecture are manually crafted, and another eight CNN architecture designs, AE-CNN+E2EPP shows the very promising results on both classification accuracy and consumed GPU days. In addition, the efficiency of the E2EPP is also examined on AE-CNN by enabling and disabling it. The results show that E2EPP could save 2/3 of computational time when achieving the same classification accuracy. Further, the E2EPP is compared with the other two state-of-the-art performance predictors, namely, FBO and Peephole, on three measurements. E2EPP shows the best results on these three measurements. Especially, E2EPP

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

uses only 1/100 computational time of that Peephole uses. It also shows that E2EPP achieves the best prediction through the qualitative comparison among these three performance predictors on both CIFAR10 and CIFAR100 datasets.

The proposed performance predictor is based on the first-train-then-to-predict paradigm, which requires a set of trained samples in advance. Theoretically, the more trained samples, the better prediction results. However, collecting more trained samples implies consuming more computational resource, which leads to long processing time. Achieving a better performance prediction results conflicts to the goal of designing the performance predictors. Therefore, finding a good balance between these two objectives may be an efficient way to use the performance predictor, which is left for future work. In addition, we will also integrate E2EPP into other CNN architecture design algorithms to solve some specific tasks, such as recognizing breast cancer on the BreakHis dataset [66].

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 25th Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[2] Y. Leung, Y. Gao, and Z.-B. Xu, "Degree of population diversity—A perspective on premature convergence in genetic algorithms and its Markov chain analysis," *IEEE Trans. Neural Netw.*, vol. 8, no. 5, pp. 1165–1176, Sep. 1997.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[4] T. N. Sainath, A.-R. Mohamed, B. Kingsbury, and B. Ramabhadran, "Deep convolutional neural networks for LVCSR," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, Vancouver, BC, Canada, 2013, pp. 8614–8618.

[5] O. Abdel-Hamid, A.-R. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 22, no. 10, pp. 1533–1545, Oct. 2014.

[6] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Montreal, QC, Canada, 2014, pp. 3104–3112.

[7] Y. Sun, G. G. Yen, and Z. Yi, "Evolving unsupervised deep neural networks for learning meaningful representations," *IEEE Trans. Evol. Comput.*, vol. 23, no. 1, pp. 89–103, Feb. 2019.

[8] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, Toulon, France, 2017, pp. 1–18.

[9] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, Toulon, France, 2017, pp. 1–16.

[10] Z. Zhong, J. Yan, and C.-L. Liu, "Practical network blocks design with q-learning," in *Proc. AAAI Conf. Artif. Intell.*, New Orleans, LA, USA, 2018.

[11] L. Xie and A. Yuille, "Genetic CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, Venice, Italy, 2017, pp. 1388–1397.

[12] E. Real *et al.*, "Large-scale evolution of image classifiers," in *Proc. Mach. Learn. Res.*, Sydney, NSW, Australia, 2017, pp. 2902–2911.

[13] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," in *Proc. Mach. Learn. Res.*, Stockholm, Sweden, 2018, pp. 1–13.

[14] M. Suganuma, S. Shirakawa, and T. Nagao, "A genetic programming approach to designing convolutional neural network architectures," in *Proc. Genet. Evol. Comput. Conf.*, Berlin, Germany, 2017, pp. 497–504.

[15] A. Krizhevsky and G. Hinton. (2009). *Learning Multiple Layers of Features From Tiny Images*. [Online]. Available: http://www.cs.toronto.edu/kriz/cifar.html

[16] T. Back, *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*. Oxford, U.K.: Oxford Univ. Press, 1996.

[17] W. Banzhaf, P. Nordin, R. E. Keller, and F. D. Francone, *Genetic Programming: An Introduction*. San Francisco, CA, USA: Morgan Kaufmann, 1998.

[18] L. M. Schmitt, "Theory of genetic algorithms," *Theor. Comput. Sci.*, vol. 259, nos. 1–2, pp. 1–61, 2001.

[19] Y. Sun, G. G. Yen, and Z. Yi, "Improved regularity model-based EDA for many-objective optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 5, pp. 662–678, Oct. 2018.

[20] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[21] Y. Sun, G. G. Yen, and Z. Yi, "Reference line-based estimation of distribution algorithm for many-objective optimization," *Knowl. Based Syst.*, vol. 132, pp. 129–143, Sep. 2017.

[22] H. Wang, L. Jiao, and X. Yao, "Two_arch2: An improved two-archive algorithm for many-objective optimization," *IEEE Trans. Evol. Comput.*, vol. 19, no. 4, pp. 524–541, Aug. 2015.

[23] Y. Sun, G. G. Yen, and Z. Yi, "IGD indicator-based evolutionary algorithm for many-objective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 23, no. 2, pp. 173–187, Apr. 2019.

[24] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "A new two-stage evolutionary algorithm for many-objective optimization," *IEEE Trans. Evol. Comput.*, to be published. doi: 10.1109/TEVC.2018.2882166.

[25] L. Bottou, "Stochastic gradient descent tricks," in *Neural Networks: Tricks of the Trade*. Heidelberg, Germany: Springer, 2012, pp. 421–436.

[26] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[27] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA, USA: MIT Press, 1998.

[28] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Evolving deep convolutional neural networks for image classification," *IEEE Trans. Evol. Comput.*, to be published. doi: 10.1109/TEVC.2019.2916183.

[29] K. Swersky, J. Snoek, and R. P. Adams, "Freeze-thaw Bayesian optimization," *CoRR*, vol. abs/1406.3896, 2014. [Online]. Available: http://arxiv.org/abs/1406.3896

[30] T. Domhan, J. T. Springenberg, and F. Hutter, "Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves," in *Proc. 24th Int. Conf. Artif. Intell.*, vol. 15. Buenos Aires, Argentina, 2015, pp. 3460–3468.

[31] A. Klein, S. Falkner, J. T. Springenberg, and F. Hutter, "Learning curve prediction with Bayesian neural networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2016, pp. 1–16.

[32] B. Deng, J. Yan, and D. Lin, "Peephole: Predicting network performance before training," *CoRR*, vol. abs/1712.03351, 2017. [Online]. Available: http://arxiv.org/abs/1712.03351

[33] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[34] R. Istrate, F. Scheidegger, G. Mariani, D. S. Nikolopoulos, C. Bekas, and A. C. I. Malossi, "TAPAS: Train-less accuracy predictor for architecture search," *CoRR*, vol. abs/1806.00250, 2018. [Online]. Available: http://arxiv.org/abs/1806.00250

[35] B. Baker, O. Gupta, R. Raskar, and N. Naik, "Accelerating neural architecture search using performance prediction," in *Proc. Int. Conf. Learn. Represent. Workshop*, 2017, pp. 1–19.

[36] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm Evol. Comput.*, vol. 1, no. 2, pp. 61–70, 2011.

[37] S. Jeong, M. Murayama, and K. Yamamoto, "Efficient optimization design method using Kriging model," *J. Aircraft*, vol. 42, no. 2, pp. 413–420, 2005.

[38] D. S. Broomhead and D. Lowe, "Radial basis functions, multi-variable functional interpolation and adaptive networks," Roy. Signals Radar Establishment, Malvern, U.K., Rep. RSRE-MEMO-4148, 1988.

[39] H. Wang, Y. Jin, and J. Doherty, "Committee-based active learning for surrogate-assisted particle swarm optimization of expensive problems," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2664–2677, Sep. 2017.

[40] Y. Jin, H. Wang, T. Chugh, D. Guo, and K. Miettinen, "Data-driven evolutionary optimization: An overview and case studies," *IEEE Trans. Evol. Comput.*, vol. 23, no. 3, pp. 442–458, Jun. 2019. doi: 10.1109/TEVC.2018.2869001.

[41] Z. Zhou, Y. S. Ong, P. B. Nair, A. J. Keane, and K. Y. Lum, "Combining global and local surrogate models to accelerate evolutionary optimization," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 37, no. 1, pp. 66–76, Jan. 2007.

[42] H. Wang, Y. Jin, and J. O. Janson, "Data-driven surrogate-assisted multiobjective evolutionary optimization of a trauma system," *IEEE Trans. Evol. Comput.*, vol. 20, no. 6, pp. 939–952, Dec. 2016.

[43] T. Chugh, N. Chakraborti, K. Sindhya, and Y. Jin, "A data-driven surrogate-assisted evolutionary algorithm applied to a many-objective blast furnace optimization problem," *Mater. Manuf. Processes*, vol. 32, no. 10, pp. 1172–1178, 2017.

[44] H. Wang, Y. Jin, C. Sun, and J. Doherty, "Offline data-driven evolutionary optimization using selective surrogate ensembles," *IEEE Trans. Evol. Comput.*, vol. 23, no. 2, pp. 203–216, Apr. 2019. doi: 10.1109/TEVC.2018.2834881.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.

[45] T. K. Ho, "Random decision forests," in *Proc. 3rd Int. Conf. Document Anal. Recognit.*, vol. 1. Montreal, QC, Canada, 1995, pp. 278–282.
[46] A. Liaw and M. Wiener, "Classification and regression by random forest," *R News*, vol. 2, no. 3, pp. 18–22, 2002.
[47] T. K. Ho, "The random subspace method for constructing decision forests," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 8, pp. 832–844, Aug. 1998.
[48] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Completely automated CNN architecture design based on blocks," *IEEE Trans. Neural Netw. Learn. Syst.*, to be published. doi: 10.1109/TNNLS.2019.2919608.
[49] K. He, X. Zhang, S. Ren, and J. Sun, *Identity Mappings in Deep Residual Networks* (Lecture Notes in Computer Science). Amsterdam, The Netherlands: Springer, 2016, pp. 630–645.
[50] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Honolulu, HI, USA, 2017, pp. 2261–2269.
[51] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. Boca Raton, FL, USA: CRC Press, 1984.
[52] H. Wang and Y. Jin, "A random forest-assisted evolutionary algorithm for data-driven constrained multiobjective combinatorial optimization of trauma systems," *IEEE Trans. Cybern.*, to be published.
[53] T. Bartz-Beielstein and M. Zaefferer, "Model-based methods for continuous and discrete global optimization," *Appl. Soft Comput.*, vol. 55, pp. 154–167, Jun. 2017.
[54] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Las Vegas, NV, USA, 2016, pp. 770–778.
[55] G. Martínez-Muñoz and A. Suárez, "Out-of-bag estimation of the optimal sample size in bagging," *Pattern Recognit.*, vol. 43, no. 1, pp. 143–152, 2010.
[56] J. Park and I. W. Sandberg, "Universal approximation using radial-basis-function networks," *Neural Comput.*, vol. 3, no. 2, pp. 246–257, 1991.
[57] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proc. 30th Int. Conf. Mach. Learn.*, Atlanta, GA, USA, Jun. 2013, pp. 1319–1327.
[58] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 1–14.
[59] M. Lin, Q. Chen, and S. Yan, "Network in network," in *Proc. Int. Conf. Learn. Represent.*, Banff, AB, Canada, 2014, pp. 1–10.
[60] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Highway networks," in *Proc. Int. Conf. Learn. Represent. Workshop*, San Diego, CA, USA, 2015, pp. 1–6.
[61] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for simplicity: The all convolutional net," in *Proc. Int. Conf. Learn. Represent.*, San Diego, CA, USA, 2015, pp. 1–14.
[62] G. Larsson, M. Maire, and G. Shakhnarovich, "Fractalnet: Ultra-deep neural networks without residuals," in *Proc. 5th Int. Conf. Learn. Represent.*, 2016, pp. 1–11. [Online]. Available: https://openreview.net/forum?id=S1VaB4cex
[63] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Stat.*, 2010, pp. 249–256.
[64] P. K. Sen, "Estimates of the regression coefficient based on Kendall's Tau," *J. Amer. Stat. Assoc.*, vol. 63, no. 324, pp. 1379–1389, 1968.
[65] D. J. Ozer, "Correlation and the coefficient of determination," *Psychol. Bull.*, vol. 97, no. 2, pp. 307–315, 1985.
[66] F. A. Spanhol, L. S. Oliveira, C. Petitjean, and L. Heutte, "A dataset for breast cancer histopathological image classification," *IEEE Trans. Biomed. Eng.*, vol. 63, no. 7, pp. 1455–1462, Jul. 2016.



Yanan Sun (S'15–M'18) received the Ph.D. degree in engineering from Sichuan University, Chengdu, China, in 2017.

He was a Research Fellow with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. He is currently a Professor (research) with the College of Computer Science, Sichuan University. His current research interests include evolutionary algorithms, deep learning, and evolutionary deep learning.

Dr. Sun is the Leading Organizer of the First Workshop on Evolutionary Deep Learning and the Special Session on Evolutionary Deep Learning and Applications in CEC19, and the Founding Chair of the IEEE CIS Task Force on Evolutionary Deep Learning and Applications.



Handing Wang (S'10–M'16) received the B.Eng. and Ph.D. degrees from Xidian University, Xi'an, China, in 2010 and 2015, respectively.

She is currently a Professor with the School of Artificial Intelligence, Xidian University. Her current research interests include nature-inspired computation, multiobjective optimization, multiple criteria decision making, surrogate-assisted evolutionary optimization, and real-world problems.

Dr. Wang is an Associate Editor of the *IEEE Computational Intelligence Magazine* and *Complex and Intelligent Systems*, and the Chair of the Task Force on Intelligence Systems for Health within the Intelligent Systems Applications Technical Committee of IEEE Computational Intelligence Society.



Bing Xue (M'10) received the B.Sc. degree from the Henan University of Economics and Law, Zhengzhou, China, in 2007, the M.Sc. degree in management from Shenzhen University, Shenzhen, China, in 2010, and the Ph.D. degree in computer science from the Victoria University of Wellington, Wellington, New Zealand, in 2014.

She is currently an Associate Professor with the School of Engineering and Computer Science, Victoria University of Wellington. She has over 100 papers published in fully refereed international journals and conferences and most of them are on evolutionary feature selection and construction. Her current research interests include evolutionary computation, feature selection, feature construction, multiobjective optimization, image analysis, transfer learning, data mining, and machine learning.

Dr. Xue is currently the Chair of the IEEE Task Force on Evolutionary Feature Selection and Construction and the IEEE Computational Intelligence Society (CIS), the Vice-Chair of the IEEE CIS Data Mining and Big Data Analytics Technical Committee, and the IEEE CIS Task Force on Transfer Learning and Transfer Optimization. She is also an Associate Editor/Editorial Board Member for five international journals and a reviewer of over 50 international journals. She is the Finance Chair of IEEE Congress on Evolutionary Computation in 2019.



Yaochu Jin (M'98–SM'02–F'16) received the B.Sc., M.Sc., and Ph.D. degrees from Zhejiang University, Hangzhou, China, in 1988, 1991, and 1996, respectively, and the Dr.-Ing. degree from Ruhr University Bochum, Bochum, Germany, in 2001.

He is currently a Professor of computational intelligence with the Department of Computer Science, University of Surrey, Guildford, U.K., where he heads the Nature Inspired Computing and Engineering Group. He is also a Finland Distinguished Professor funded by the Finnish Agency for Innovation (Tekes) and a Changjiang Distinguished Visiting Professor appointed by the Ministry of Education, China. He has (co)authored over 200 peer-reviewed journal and conference papers and has been granted eight patents on evolutionary optimization. His current research interests include computational intelligence, computational neuroscience, and computational systems biology. He is particularly interested in nature-inspired, real-world-driven problem solving.

Dr. Jin was a recipient of the 2014 and 2016 *IEEE Computational Intelligence Magazine* Outstanding Paper Award and the 2017 IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION Outstanding Paper Award. He is the Editor-in-Chief of the IEEE TRANSACTIONS ON COGNITIVE AND DEVELOPMENTAL SYSTEMS and *Complex and Intelligent Systems*. He has been an IEEE Distinguished Lecturer since 2017. He was the Vice President for Technical Activities of the IEEE Computational Intelligence Society from 2014 to 2015.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.



Gary G. Yen (S'87–M'88–SM'97–F'09) received the Ph.D. degree in electrical and computer engineering from the University of Notre Dame, Notre Dame, IN, USA, in 1992.

He was with the Structure Control Division, U.S. Air Force Research Laboratory, Albuquerque, NM, USA. He is currently a Regents Professor with the School of Electrical and Computer Engineering, Oklahoma State University, Stillwater, OK, USA. His current research interests include intelligent control, computational intelligence, conditional health monitoring, signal processing, and their industrial/defense applications.

Dr. Yen was a recipient of the Andrew P. Sage Best Transactions Paper Award from IEEE Systems, Man and Cybernetics Society in 2011 and the Meritorious Service Award from IEEE Computational Intelligence Society in 2014. He was an Associate Editor of the *IEEE Control Systems Magazine*, the IEEE TRANSACTIONS ON CONTROL SYSTEMS TECHNOLOGY, *Automatica*, *Mechantronics*, the IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS—PART A: SYSTEMS AND HUMANS, the IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS—PART B: CYBERNETICS, and the IEEE TRANSACTIONS ON NEURAL NETWORKS. He is currently serving as an Associate Editor for the IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION and the IEEE TRANSACTIONS ON CYBERNETICS. He served as the General Chair for the 2003 IEEE International Symposium on Intelligent Control held in Houston, TX, USA, and the 2006 IEEE World Congress on Computational Intelligence held in Vancouver, BC, Canada. He served as the Vice President for the Technical Activities in 2005–2006 and then the President in 2010–2011 of the IEEE Computational Intelligence Society. He was the Founding Editor-in-Chief of the *IEEE Computational Intelligence Magazine* from 2006 to 2009.



Mengjie Zhang (M'04–SM'10–F'18) received the B.E. and M.E. degrees from the Artificial Intelligence Research Center, Agricultural University of Hebei, Hebei, China, in 1989 and 1992, respectively, and the Ph.D. degree in computer science from RMIT University, Melbourne, VIC, Australia, in 2000.

He is currently a Professor of computer science, the Head of the Evolutionary Computation Research Group, and an Associate Dean (Research and Innovation) with the Faculty of Engineering, Victoria University of Wellington, Wellington, New Zealand. He has published over 350 research papers in refereed international journals and conferences. His current research interests include evolutionary computation, particularly genetic programming, particle swarm optimization, and learning classifier systems with application areas of image analysis, multiobjective optimization, feature selection and reduction, job shop scheduling, and transfer learning.

Prof. Zhang is currently the Chair of the IEEE CIS Intelligent Systems and Applications Technical Committee, and the immediate Past Chair of the IEEE CIS Emergent Technologies Technical Committee and the Evolutionary Computation Technical Committee, and a member of the IEEE CIS Award Committee. He is the Vice-Chair of the IEEE CIS Task Force on Evolutionary Feature Selection and Construction and the Task Force on Evolutionary Computer Vision and Image Processing, and the Founding Chair of the IEEE Computational Intelligence Chapter in New Zealand. He is also a Committee Member of the IEEE NZ Central Section. He is a fellow of the Royal Society of New Zealand and have been a Panel Member of the Marsden Fund (New Zealand Government Funding). He is also a member of ACM.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on October 19,2026 at 01:52:26 UTC from IEEE Xplore. Restrictions apply.